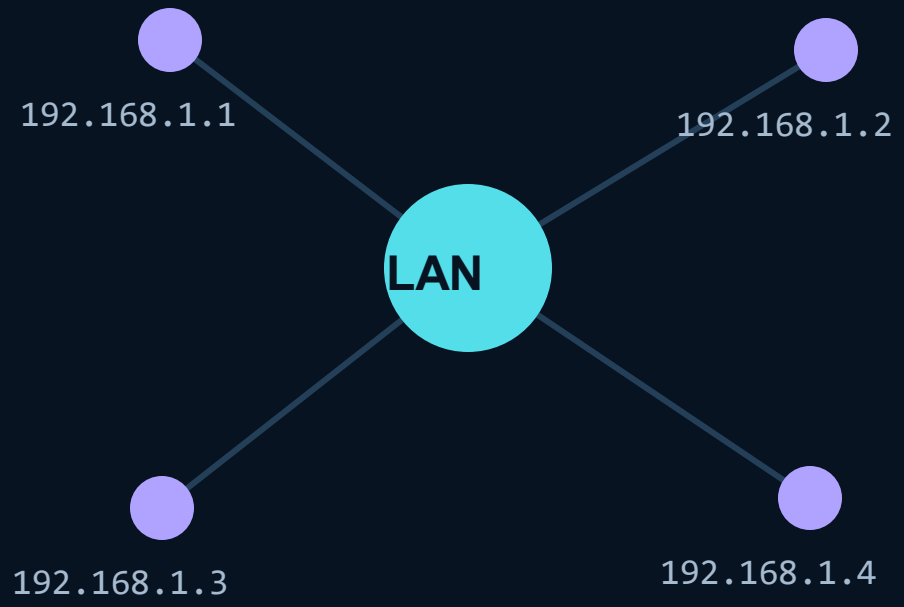
SubnetScope

Automated Subnet Reconnaissance
and Port Status Profiling

Understand the subnet.
Discover the hosts.
Interpret the ports.

An interactive learning lab with a separate, restricted local TCP utility.

WEBSITE DEMO + AUTHORIZED LOCAL CLI



Illustrative private network

Why does this project matter?

A growing network is difficult to understand by checking every device manually.

The manual task

Find the address range.
Check candidate devices.
Try service connections.
Record each outcome.

The automated workflow

Validate once.
Calculate the subnet.
Run bounded checks.
Classify and summarize.

Objective: turn authorized network checks into a repeatable, understandable workflow.

Start with four simple ideas

Example: a small organizational LAN using private IPv4 addresses.

Network

Connected devices that exchange data.

Host

A device taking part in the network.

IP address

An address for a network interface.

TCP port

A numbered service connection endpoint.

192.168.1.2 : 443 = sample host address + TCP service endpoint

What does /24 mean?

An IPv4 address has 32 bits. CIDR tells us how many belong to the network.

192.168.1.0/24



24 network bits / subnet mask: 255.255.255.0

8 host bits

2^8 = 256

Total IPv4 addresses

256 - 2 = 254

Conventional usable hosts

Network + broadcast

Two reserved addresses

Special cases: /31 has 2 point-to-point endpoints; /32 represents 1 address.

Calculate a small subnet

The lab example is 192.168.1.0/29: small enough to follow every address.

32 - 29 = 3 host bits **2^3 = 8 total** **8 - 2 = 6 usable**

. 0	. 1	. 2	. 3	. 4	. 5	. 6	. 7
Network	Usable host	Usable host	Usable host	Usable host	Usable host	Usable host	Broadcast

`Mask 255.255.255.248 | Usable range 192.168.1.1 - 192.168.1.6`

Total addresses and usable hosts are different quantities.

Follow the complete workflow

Each stage produces structured information for the next stage.

01 Authorized input
Choose an allowed CIDR.

>

02 Validation
Check syntax and scope.

>

03 Subnet calculation
Find range and counts.

>

04 Candidates
List eligible addresses.

05 Discovery
Record probe responses.

>

06 TCP profiling
Try selected TCP ports.

>

07 Classification
Interpret each outcome.

>

08 Report
Summarize the evidence.

Website: illustrative discovery. Local CLI: bounded TCP checks of every allowed candidate.

Candidate, responsive host, or service?

These are three different levels of evidence.

1 Candidate address

Inside the chosen range.
Eligible for a check.
No response proven yet.

2 Responsive host

Answered a selected
host discovery check.
Evidence for that check.

3 TCP endpoint

Accepted a connection
on one selected port.
Service identity unproven.

No response does not prove that a machine is offline.

How do we classify a TCP port?

A normal TCP connection attempt provides an outcome for one address and port.

OPEN

Connection accepted.

CLOSED

Connection actively refused.

FILTERED / NO RESPONSE

No conclusive reply before timeout.

ERROR / UNKNOWN

Another error prevented classification.

Conventions: 22 SSH | 53 DNS (TCP also) | 80 HTTP | 443 HTTPS

A timeout cannot identify the cause. A port number cannot identify the application.

Architecture matches the implementation

The public learning experience and real TCP checks are separate.

PUBLIC WEBSITE / SIMULATION

React interface > IPv4 calculator > Sample events > Classifications > Lab summary

LOCAL NODE.JS CLI / REAL TCP

CIDR + ports > Safety validation > TCP sockets > Classifications > Table + JSON

Shared IPv4 logic; separate data sources. Real scan results stay on the local computer.

Demonstrate the two lab tabs

All port-lab events and counts below are simulated.

Tab 1 Subnet Explorer

Calculate 192.168.1.0/24.
Change to /29; inspect the range.
Try /31 or /32.
Enter invalid input to see validation.

Tab 2 Port Profiling Lab

Start the deterministic demo.
Inspect an individual result.
Filter and sort the table.
Reset to the initial state.

36 simulated probes = 6 open + 17 closed + 12 no response + 1 unknown

Real testing stays in an authorized lab

The CLI validates the entire scan plan before creating any network connections.

14
Maximum candidates

16
Maximum concurrency

100-2000 ms
Bounded timeout

`Allowlist: 22, 53, 80, 443, 3306, 5432, 8080`

```
npm run scan -- --cidr 127.0.0.1/32 --ports 80,443 --dry-run

npm run scan -- --cidr 127.0.0.1/32 --ports 80,443 --authorized
```

Permission is required. Output is a local table and optional local JSON; the website never scans.

What did we build and verify?

SubnetScope connects networking theory to a working, understandable project.

24 tests passed

Subnet boundaries, validation and CLI safety.

13 browser groups

Core journeys, responsive layouts and WebGL fallback.

Build checks passed

Lint, strict TypeScript, production build.

Limits: no response is inconclusive; a port suggests a service; an open connection does not establish that a service is secure.

Conclusion: calculate accurately, test only authorized targets, and interpret the evidence carefully.